

Final Project Specification

SYSTEM MONITORING

Author: Phan Vu Huynh Tuan

Document History

Date	Version	Author	Description
04.10.2025	V1.0	Phan Vu Huynh Tuan	Initial version for the specification

Contents

1.	Project Introduction.....	4
2.	System Architecture Overview.....	5
3.	Project Folder Structure.....	6
4.	Driver Specification.....	7
4.1.	Driver Architecture.....	7
4.2.	Device Tree.....	8
5.	Application Specification.....	10
5.1.	Application Architecture.....	10
5.2.	Main window.....	11
5.3.	CPU window.....	13
5.4.	Memory window.....	14
5.5.	Time window.....	16

1. Project Introduction

The intended project is to build up a real-time monitoring and analysis application for the performance of a development device (e.g. Raspberry Pi, Beagle Bone...).

The system tracks key resource such as CPU usage, memory consumption, network bandwidth and other critical system metrics.

The purpose of this application is to provide continuous monitoring of system resources and help maintain system stability through a dedicated display interface.

For the application described in this document, below components are selected:

- Hardware:
 - Development board: Raspberry Pi Zero 2W.
 - Screen: LCD TFT 2.8" ILI9341 (w/o touch)
 - Other components: Customer mica case, mini breadboard.
- Operating system: Raspberry Pi OS, kernel version: 6.12.25
- QT Framework: Version 6.4.2

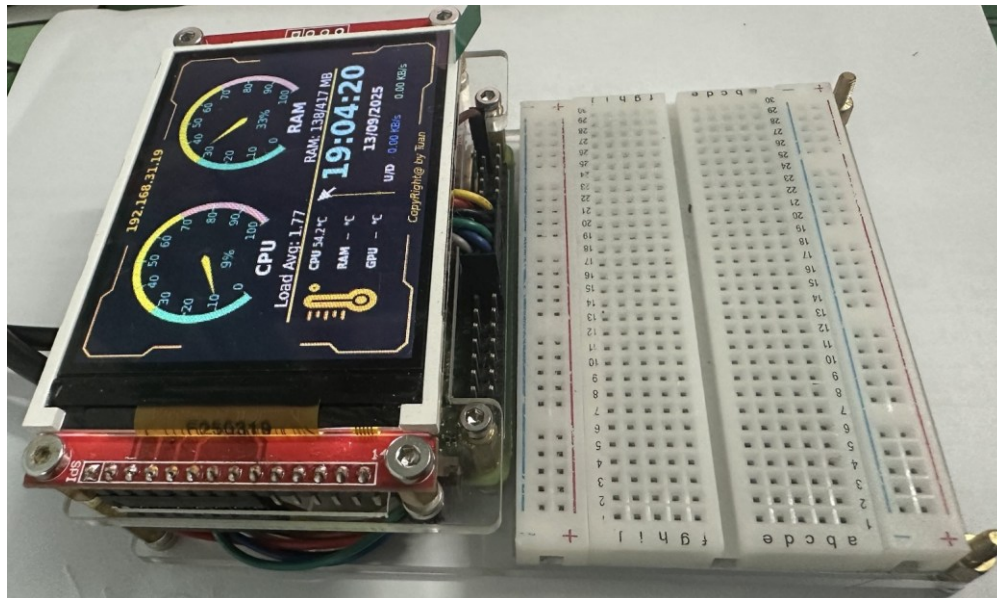


Figure 1. Final Product

2. System Architecture Overview

In this application, the system is separated to 2 parts: Driver for screen and application (Figure 2):

- The Driver is built based on the framebuffer driver, enabling applications in user space to communicate with the hardware (LCD) via the SPI bus. In this setup, two physical devices are used:
 - SPI – handles data transfer.
 - GPIO – controls reset and switches between command/data modes for the LCD.
- The Application is developed using the Qt framework, which provides developers with a user-friendly environment and simplified tools for building GUI applications.

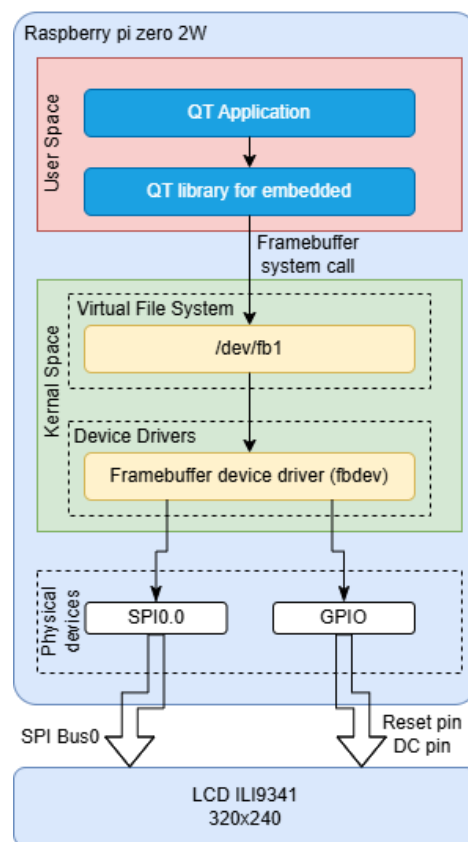


Figure 2. The system architecture

The Driver is developed as framebuffer device, which let the application based on QT framework easily to work.

3. Project Folder Structure

This repository contains all files and resources for the Linux Final Project, developed for the Raspberry Pi. It is organized into several structured directories to separate documents, design files, source code, and hardware resources.

The following section describes the purpose and contents of each folder in detail:

LinuxFinalProject	
----- .vscode	# Contains FTP configuration for the host (Raspberry Pi)
----- 00_Common	# Contains common documents, images, and shared resources
----- 01_ArchitectureDesign	# Contains system architecture design files
----- 02_Documents	# Contains project documentation
----- 03_Code	# Contains all source code
----- 00_Driver	# Driver source code
----- 01_Application	# Application source code, including testing and QT application
----- QTProject	# Qt application source code
----- inc	# Header files
----- qrc	# Qt resource files
----- img	# Images used by the Qt resource files
----- src	# Source files (.cpp)
----- ui	# UI layout files
----- 02_DeviceTree	# Device tree files for the driver
----- 04_RPi0_Case	# Contains the Raspberry Pi Zero case drawings

Description:

- **.vscode:** Contains Visual Studio Code workspace settings and FTP configuration used to synchronize files between the development machine and the Raspberry Pi host.
- **00_Common:** Stores shared materials such as reference images, icons, and document templates used throughout the project.
- **01_ArchitectureDesign:** Includes system architecture documentation, block diagrams, and design notes describing the software and hardware interaction.
- **02_Documents:** Holds all documentation, including project reports, specifications, installation guides, and testing results.
- **03_Code:** The core section of the project, containing all software components:
 - **00_Driver** – Implements low-level hardware drivers to interface with Raspberry Pi peripherals.
 - **01_Application** – Contains high-level application code and Qt-based GUI for system interaction.
 - **02_DeviceTree** – Provides device tree source files that define hardware configurations for the Linux kernel.
- **04_RPi0_Case:** Contains 3D design files and mechanical drawings of the Raspberry Pi Zero case, used for enclosure design and fabrication.

4. Driver Specification

4.1. Driver Architecture

The driver creates a framebuffer device file then able to transfer the data from user space to physical device (SPI). So, there are two components including in the driver: Framebuffer Handling and Data Transfer Handling.

Framebuffer Handling: The primary function of framebuffer handling is to receive data from user space through system calls, store or modify it in virtual memory, and then trigger deferred I/O transfers.

- Virtual memory size is determined by the maximum pixel resolution (320×240) multiplied number of bytes per pixel (2 bytes). The result is then rounded up to match the kernel memory page size.
- The deferred I/O transfer delay is set to 20ms.

Data Transfer Handling: This component provides the functions to configure the SPI device and initiate data transfers to the LCD.

- SPI speed: 36 MHz
- SPI mode: 0
- SPI transfer via DMA: Not used

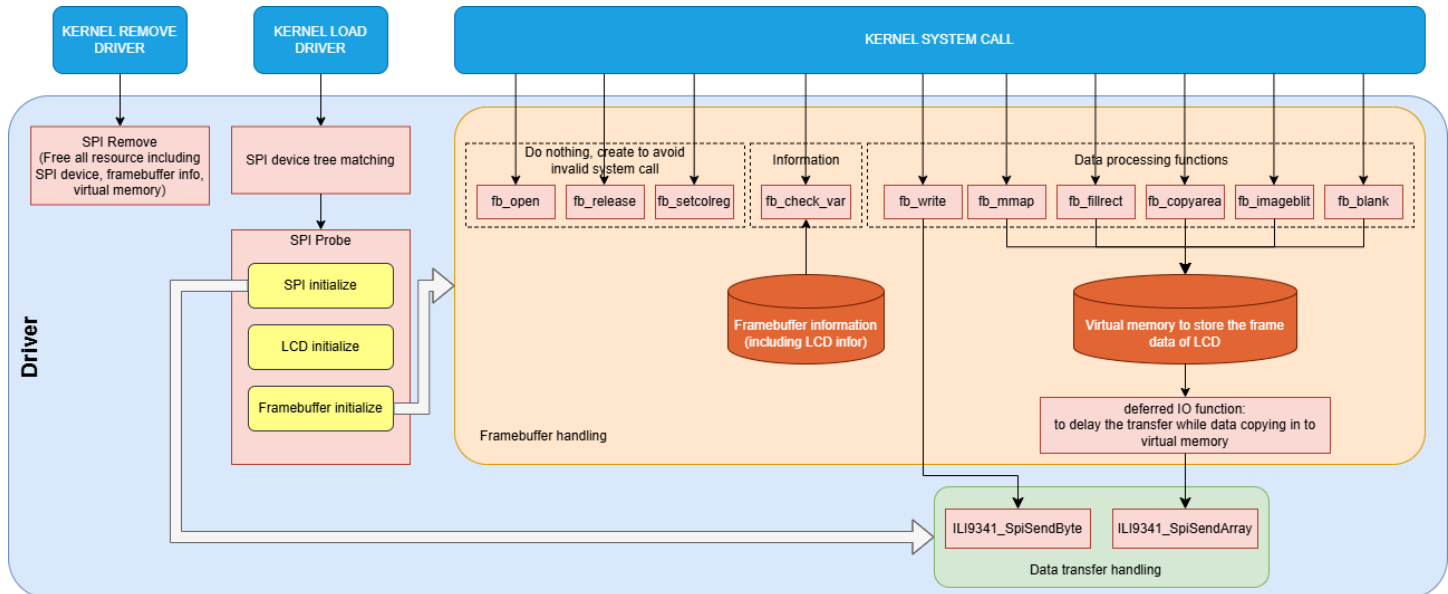


Figure 3. The driver architecture

The above diagram illustrates the driver architecture:

- The driver begins by probing the SPI device (using the compatible entry from the device tree).
- If the SPI device is successfully probed:
 - The SPI device is initialized.

- The LCD is then configured with appropriate settings, such as size, orientation, color format, and charge pump.
- Once the LCD is ready for use, the framebuffer device is created, which includes virtual memory, device information, and system call functions.
- While the driver is running, user space applications can transmit data through the framebuffer file (/dev/fb1). This file is used by the QT application, which will be described in detail in the following section.
- To stop or remove the driver, an “SPI remove” function is provided. This function releases all allocated resources to ensure no memory leaks occur.

4.2. Device Tree

```

4  / {
5      compatible = "brcm,bcm2837";
6
7      fragment@0 {
8          target = <&spi0>;
9          __overlay__ {
10             status = "okay";
11             #address-cells = <1>;
12             #size-cells = <0>;
13
14             ili9341@0 {
15                 compatible = "custom,ili9341";
16                 reg = <0>;
17                 spi-max-frequency = <40000000>;
18                 reset-gpios = <&gpio 25 0>; /* GPIO25 for RST */
19                 dc-gpios = <&gpio 24 0>; /* GPIO24 for DC */
20                 buswidth = <8>;
21
22                 // Required if you need DMA
23                 dma-coherent;
24
25                 status = "okay";
26             };
27         };
28     };
29
30     fragment@1 {
31         target = <&gpio>;
32         __overlay__ {
33             ili9341_pins: ili9341_pins {
34                 brcm,pins = <24 25>; /* GPIO24 (DC) and GPIO25 (RST) */
35                 brcm,function = <1>; /* Output */
36                 brcm,pull = <0>; /* No pull */
37             };
38         };
39     };
40 };
41

```

Figure 4. The source code of device tree

The device tree has 2 parts:

- fragment@0 (SPI): Defines a custom node ili9341@0, created specifically for the LCD device. The compatible field is set to "customer,ili9341".
 - Select SPI0
 - Max frequency: 40MHz

- Describe the GPIO24 and GPIO25 for DC and RST pin.
- Bus width: 8-bit
- fragment@1 (GPIO): Configures the GPIO pins used to control the RST and DC signals.
 - GPIO24: Configured as output for DC pin, to control the command/data transaction.
 - GPIO25: Configured as output for Reset pin, to perform a hard reset the LCD.

Note: To prevent the default Raspberry Pi OS driver from being loaded automatically, avoid setting the compatible field to "ili9341".

5. Application Specification

5.1. Application Architecture

To build the application, the following development process is defined to meet the project requirements (Figure 5):

- Install the Qt framework on both the target device (Raspberry Pi) and the development computer (Windows OS).
- On Windows, design, build, and test the UI (using simulated/random data for validation).
- Once the UI is verified, begin implementing the features for the target device. Use macros (Q_OS_LINUX and Q_OS_WIN) to separate platform-specific code.
- Repeat steps 2 and 3 until all required windows and functionalities are complete.
- Perform a final optimization phase to improve code readability, structure, and design, as well as to enhance CPU and memory efficiency.

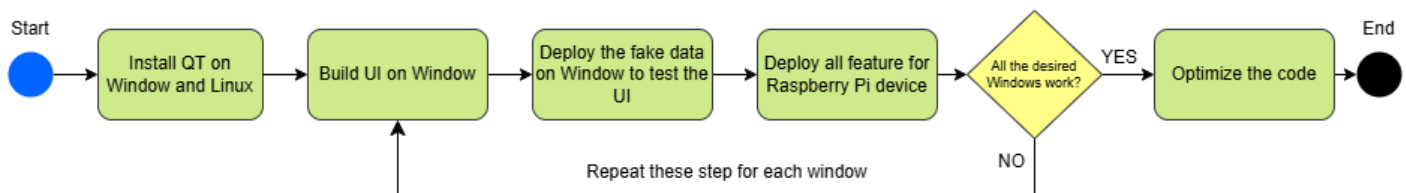


Figure 5. Application develop flow

Framework used: In this system, Qt Widgets is chosen as the UI framework instead of Qt Quick because:

- It is lightweight, making it suitable for applications running on limited CPU/GPU resources.
- It does not require hardware acceleration (GPU) to function effectively.

Application Architecture (Figure 6): The application consists of four windows:

- Main Window: Displays most of the system data using compact components such as meters and labels.
- CPU Window: Provides a detailed graph of sub-CPU usage.
- Memory Window: Shows detailed information about RAM and storage utilization.
- Time Window: Presents the current time with a visually distinctive colored background.

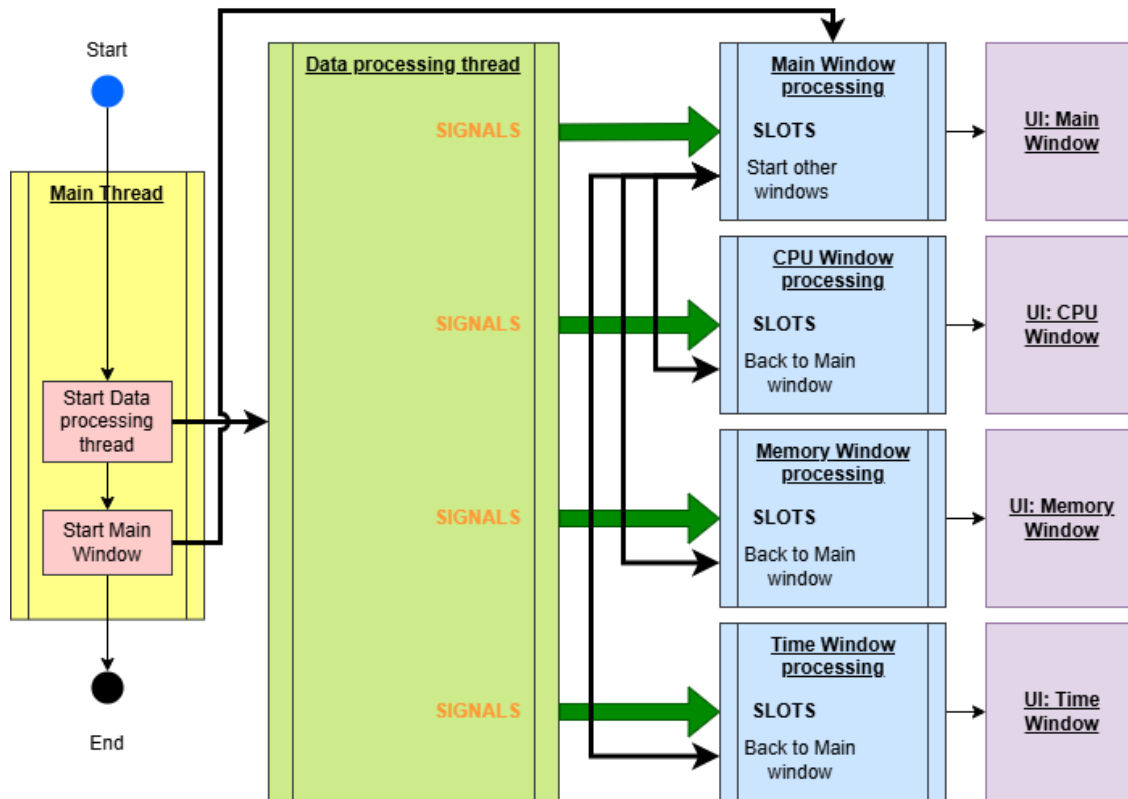


Figure 6. Application Architecture

The application runs with two threads:

- **Main Thread:** Manages the entire application, including the UI and window logic.
 - **Data Processing Thread:** Handles computation of essential system data such as CPU usage, memory usage, temperature, and network bandwidth.
- ⇒ The data processing thread communicates with the main thread via signals and slots to update the UI in real time.

5.2. Main window

The Main Window (Figure 7) is the startup window of the application. It presents the most essential system information in a clear and user-friendly layout, including:

- (1) The device's IP address displayed at the top of the window.
- (2) Two meters indicating CPU and RAM usage, they are customer "QWidget" object.
- (3) Temperature information shown at the bottom-left corner (currently limited to CPU temperature).
- (4) Date/time and network bandwidth displayed at the bottom-right corner.

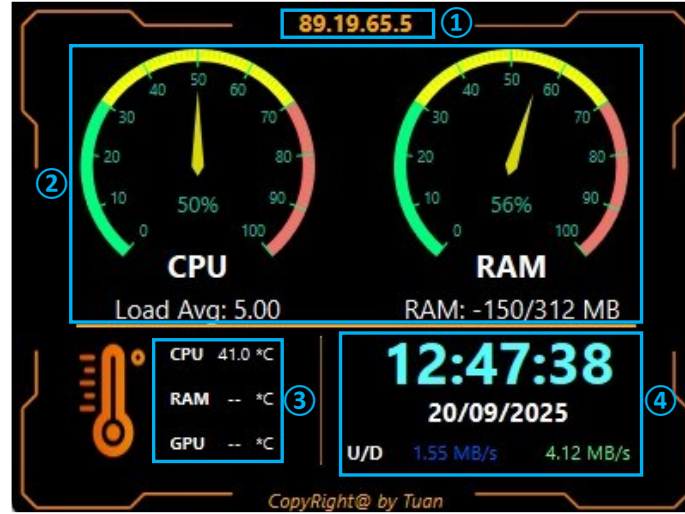


Figure 7. Main Window Layout

The Main Window receives data through signals emitted by the Data Processing Thread:

- **SIGNALS:** CPU Data Ready → **SLOTS:** CPU usage information update.
- **SIGNALS:** RAM Data Ready → **SLOTS:** RAM usage information update.
- **SIGNALS:** Network Data Ready → **SLOTS:** Network data update.
- **SIGNALS:** CPU Temperature Data Ready → **SLOTS:** CPU temperature update.

The corresponding data is then refreshed and displayed in the relevant UI components.

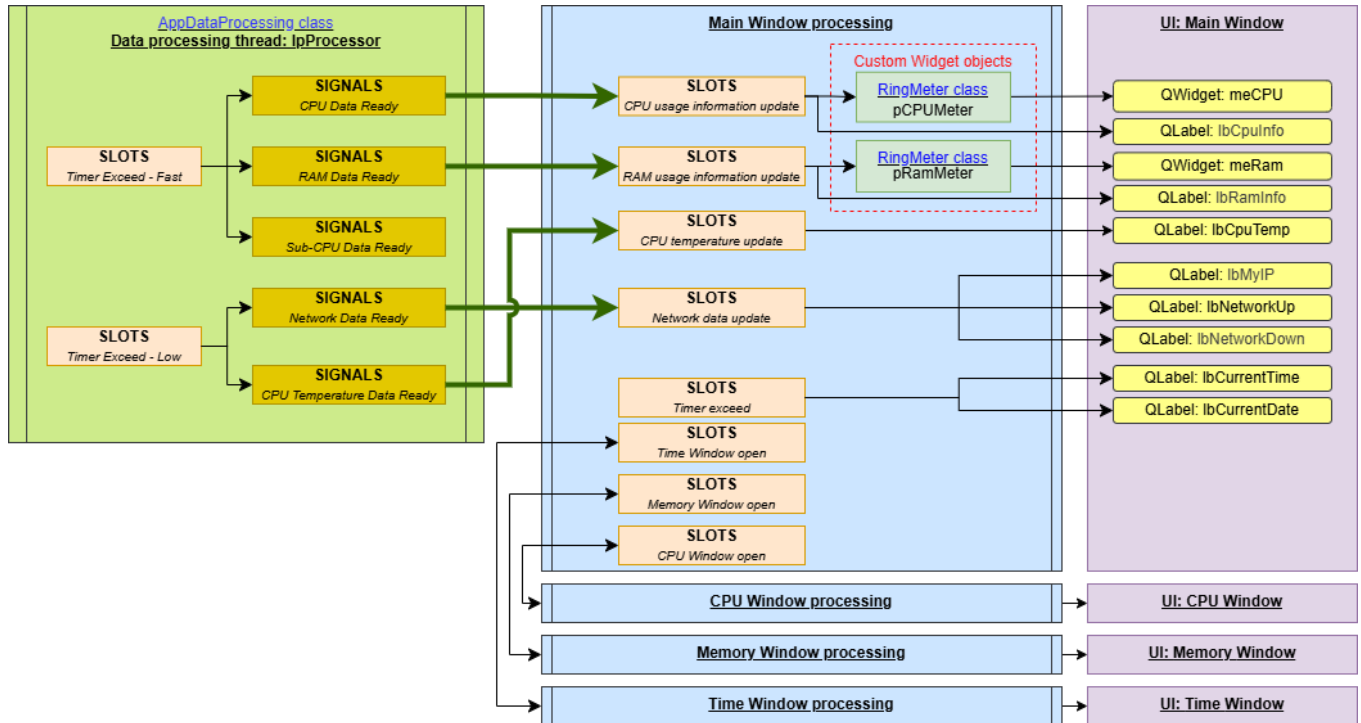


Figure 8. Main Window detail design

To switch to another window, a left mouse click can be used on the CPU usage meter, RAM usage meter, or date/time display. Each item opens its corresponding window: CPU, Memory, or Time.

5.3. CPU window

The CPU Window (Figure 9) is launched when the CPU meter in the Main Window is clicked. It displays detailed information about individual CPU cores and overall CPU performance, including:

- (1) A line graph showing the usage of individual CPU cores, with the x-axis representing time and the y-axis representing usage percentage (0–100%).
- (2) A Back button that returns to the Main Window.
- (3) The overall CPU usage.
- (4) The load average of CPU.
- (5) The CPU temperature.

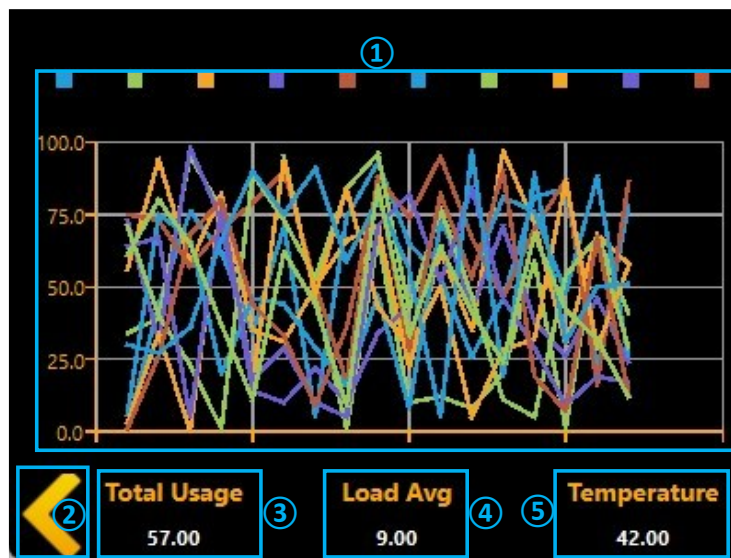


Figure 9. CPU Window layout

The CPU Window receives data through signals emitted by the Data Processing Thread:

- **SIGNALS:** CPU Data Ready → **SLOTS:** CPU usage data update.
- **SIGNALS:** Sub CPU Data Ready → **SLOTS:** Sub CPU usage data update.
- **SIGNALS:** CPU Temperature Data Ready → **SLOTS:** CPU temperature update.

The corresponding data is then refreshed and displayed in the relevant UI components.

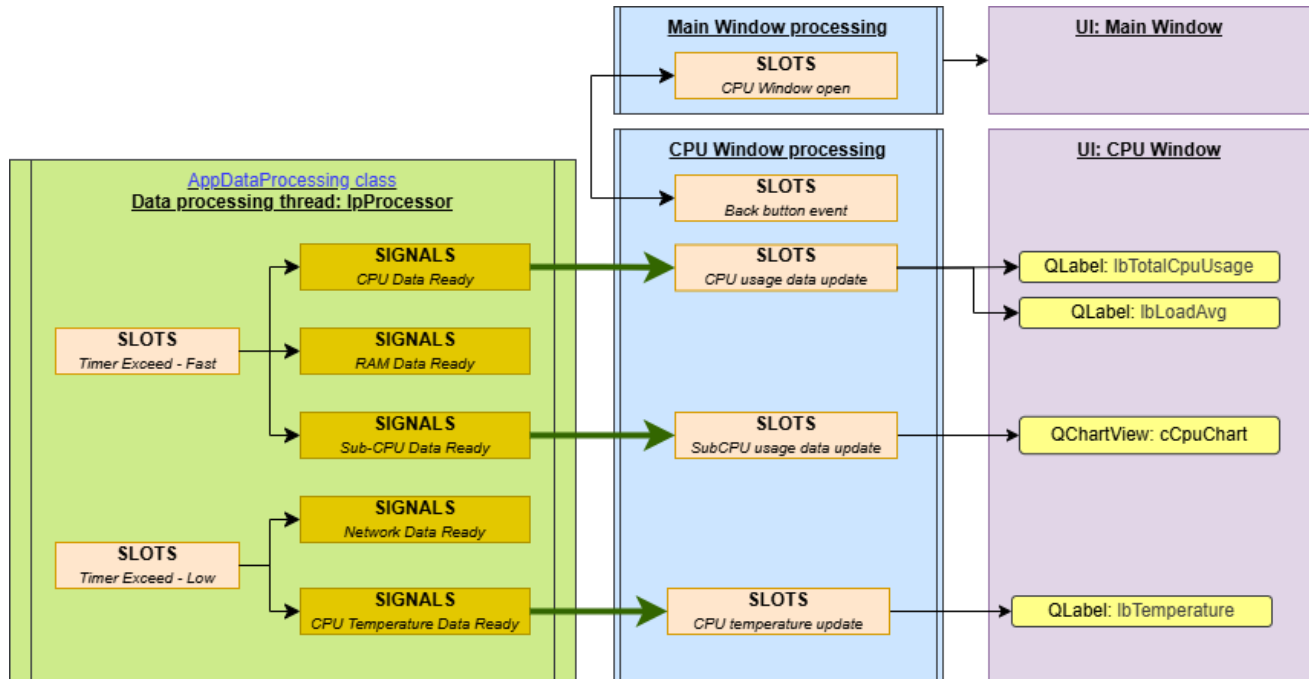


Figure 10. CPU Window detail design

To return to the Main Window, an event is assigned to the Back button. When the button is clicked, the application hides the CPU Window and reactivates the Main Window.

Note: The CPU Window is not terminated upon closing to ensure smooth reopening in future use. It is permanently terminated only when the application itself is closed, at which point all associated resources are released through the destructor.

5.4. Memory window

The Memory Window (Figure 11) is launched when the RAM meter in the Main Window is clicked. It presents detailed memory information, including:

- (1) A custom QWidget displaying the RAM usage percentage.
- (2) A custom QWidget displaying the storage usage percentage.
- (3) Detailed RAM information, including both RAM and swap memory data.
- (4) Storage details, such as the root path, device name, file system type, and space usage information.

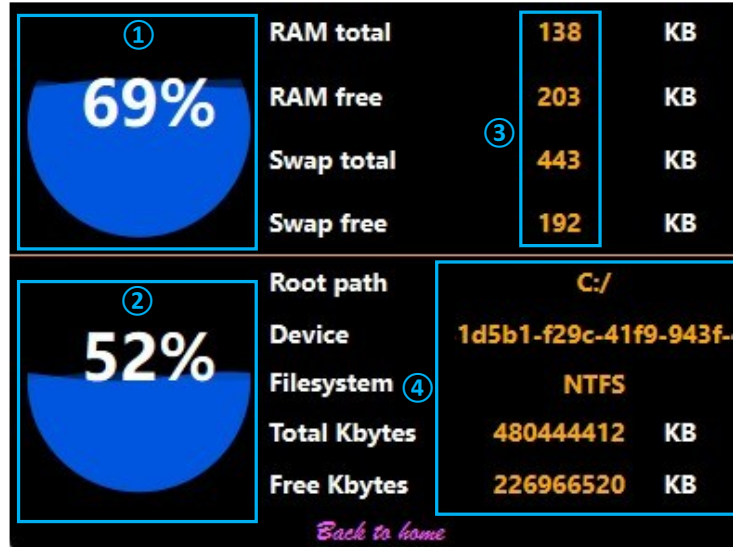


Figure 11. Memory Window layout

The Memory Window receives data through signals emitted by the Data Processing Thread:

- **SIGNALS:** RAM Data Ready → **SLOTS:** RAM usage information update.

The corresponding data is then refreshed and displayed in the relevant UI components. The storage information is calculated by the QT library within the logic.

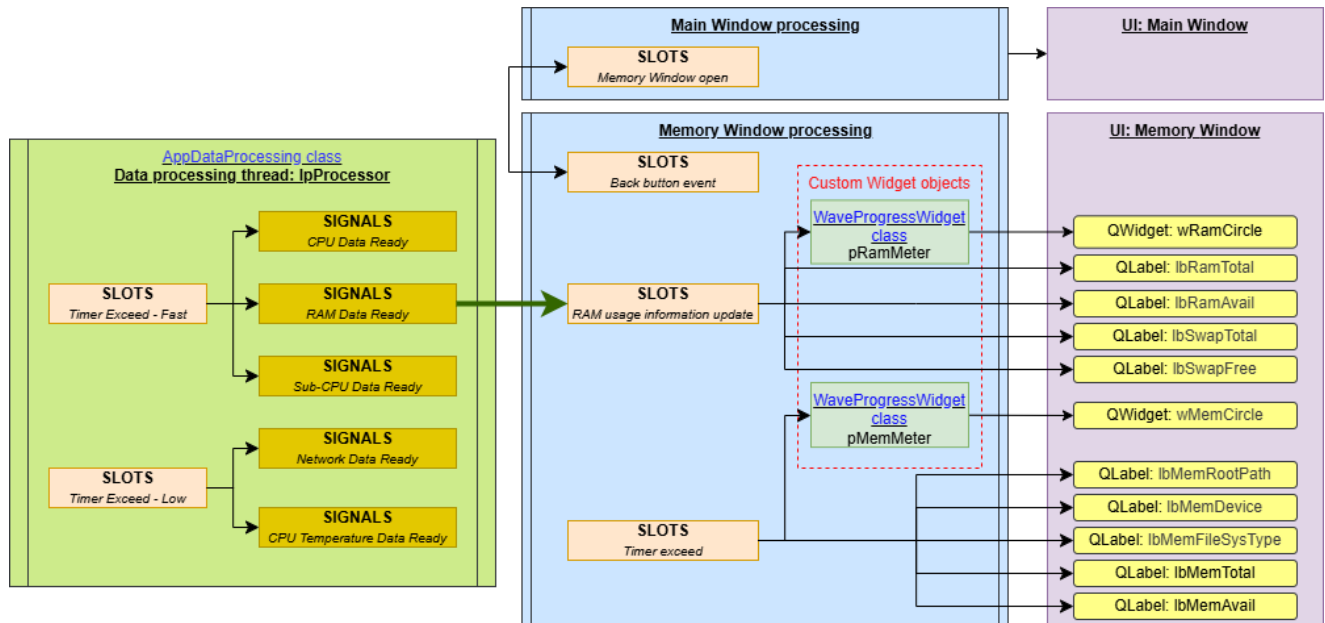


Figure 12. Memory Window detail design

To return to the Main Window, an event is assigned to the “Back to home” button. When the button is clicked, the application hides the Memory Window and reactivates the Main Window.

Note: The Memory Window is not terminated upon closing to ensure smooth reopening in future use. It is permanently terminated only when the application itself is closed, at which point all associated resources are released through the destructor.

5.5. Time window

The Time Window (Figure 13) is launched when the Time label in the Main Window is clicked. Its main purpose is to display the current time with an appealing background, along with brief system information for CPU, RAM, and network usage:

- (1) Date and time display in a large, readable format.
- (2) The simple custom progress bars showing CPU and RAM usage.
- (3) Network bandwidth information.
- (4) A Back button allowing the user to return to the Main Window.



Figure 13. Time Window layout

The Memory Window receives data through signals emitted by the Data Processing Thread:

- **SIGNALS:** CPU Data Ready → **SLOTS:** CPU usage information update.
- **SIGNALS:** RAM Data Ready → **SLOTS:** RAM usage information update.
- **SIGNALS:** Network Data Ready → **SLOTS:** Network data update.

The corresponding data is then refreshed and displayed in the relevant UI components.

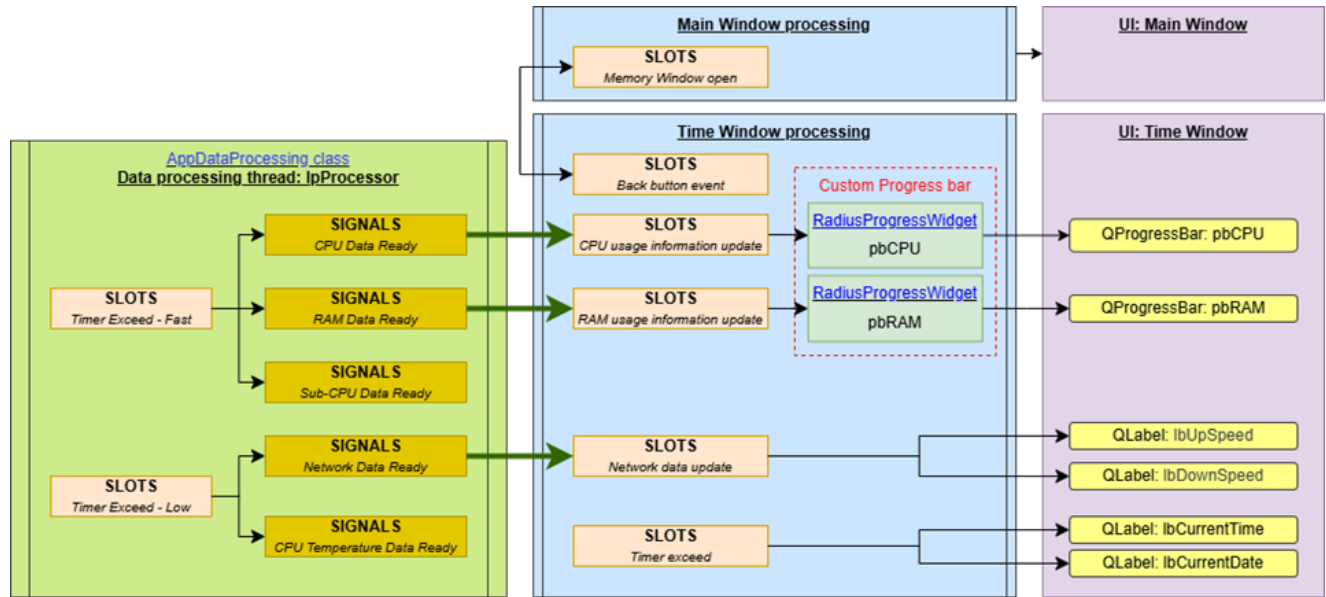


Figure 14. Time Window detail design

To return to the Main Window, an event is assigned to the “Back to home” button. When the button is clicked, the application hides the Time Window and reactivates the Main Window.

Note: The Time Window is not terminated upon closing to ensure smooth reopening in future use. It is permanently terminated only when the application itself is closed, at which point all associated resources are released through the destructor.